CMSE 801 - Day 22

Monte-Carlo Methods

Nov 23, 2022

General Course Announcements

- Homework #5 is due Wednesday Nov 30 at 11:59 p.m.
- Project's deadline Sunday Dec 4 at 11:59 p.m.
- One on one meeting next week!

Questions/comments from Day 22 PCA

- If we want to get accurate result, how many times at least should we repeat for MC methods?
- More explanation of what Monte Carlo is/more examples
- Could you please review how to do the calculate_distances function in class?
- Can you go over the last question again / the last question of the pre class
- I think just computing more approximations
- Are there any other methods to solve such problems?
- No, I think I just really like actually computing this approximations because i can see actually how the method works

In [1]:
```
import numpy as np
%matplotlib inline
import matplotlib.pyplot as plt
from IPython.display import display, clear_output
import time
import random
```

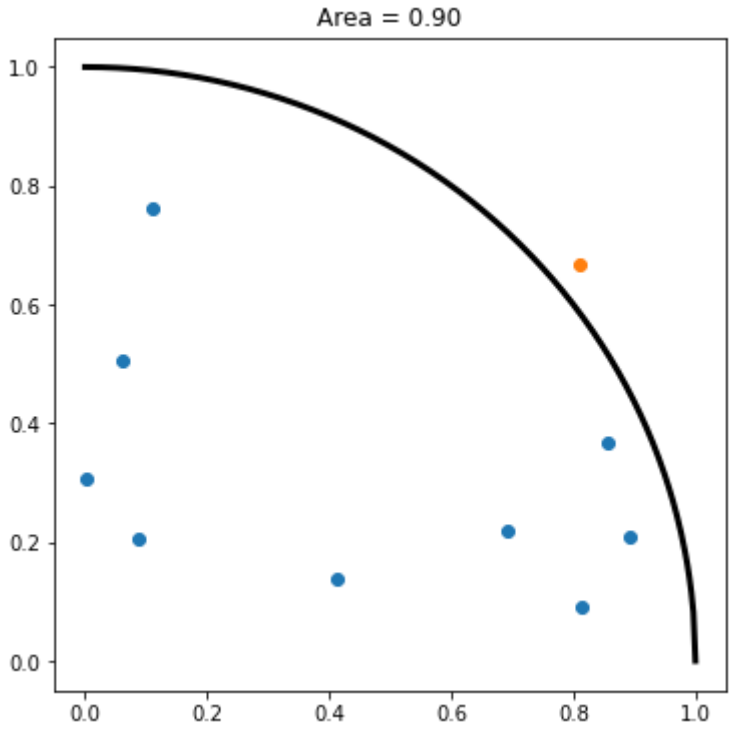
In [2]:
```
npoints = 10
is_in = 0
is_out = 0
x_in = []
y_in = []
x_out = []
y_out = []
```

In [3]:
```
fig = plt.figure(figsize=(6,6))
for i in range(npoints):
    x = random.random()
    y = random.random()

    if y < np.sqrt(1-x**2):
        is_in += 1
        x_in.append(x)
        y_in.append(y)
```

```
    else:
        is_out += 1
        x_out.append(x)
        y_out.append(y)

    plt.scatter(x_in,y_in)
    plt.scatter(x_out,y_out)
    x = np.linspace(0,1,300)
    y = np.sqrt(1-x**2)
    plt.plot(x,y,lw=3,c='k')
    plt.title("Area = %0.2f" %(is_in/npoints))
    time.sleep(0.1)
    clear_output(wait = True)
    display(fig)            # Reset display
    fig.clear()
```




```
<Figure size 432x432 with 0 Axes>
```

In [4]:
```
print("Area =", is_in/npoints)
print("Real area =", (np.pi)/4.0)
```

```
Area = 0.9
Real area = 0.7853981633974483
```

In [5]:
```
def create_cities(n_city):
    x_loc = np.random.random(n_city)
    y_loc = np.random.random(n_city)
    return x_loc,y_loc

n_city = 3
x,y = create_cities(n_city)
```

In [6]:
```
def calculate_distances(x_loc, y_loc):
    D = np.zeros((len(x_loc), len(y_loc)))
    for i in range(D.shape[0]):
        this_x = x_loc[i]
        this_y = y_loc[i]
```

```
        for j in range(D.shape[1]):
            D[i,j] = np.sqrt((x_loc[j]-this_x)**2 + (y_loc[j]-this_y)**2)
    return D
```

In [7]:
```
print(calculate_distances(x, y))
```

```
[[0.         0.69378416 0.27250031]
 [0.69378416 0.         0.71215925]
 [0.27250031 0.71215925 0.        ]]
```